

LETTER

Formal and Empirical Analysis of Authorization Decision Inconsistency in SAGA

TODO AUTHOR[†], ??

SUMMARY We analyze an order-dependent authorization decision inconsistency in SAGA. Direct matcher observations contradict documented most-specific rule semantics. In a fixed divergence scenario, ProVerif finds symbolic message acceptance reachable with prior token issuance. Controlled experiments separately trace production authorization logic to token issuance and storage, stopping before message acceptance.

key words: *authorization, access control, formal verification, AI agents, ProVerif*

1. Introduction

SAGA [1] governs communication between registered agents using contact policies, identity checks, and access-control tokens. Authentication establishes identity, not compliance with the receiver's contact policy.

Direct matcher calls show order dependence: specific denial followed by wildcard permission returns permission, whereas the reverse order returns denial; most-specific semantics deny both. For the fixed specification-deny/implementation-allow case, ProVerif reports `ChatAccept` reachable and its correspondence to prior `TokenIssue` true. A separate controlled production-consumer execution reaches token issuance/storage. The formal and empirical endpoints are symbolic message acceptance and token storage, respectively.

Our contributions are (1) analysis of the recorded matcher inconsistency, (2) consumer-side propagation validation with A/A, D/D, and D/A controls, and (3) conditional formal analysis of reachability and prior issuance.

(1) Related work.

Authorization-policy work studies policy semantics and analysis [2], [3]; ProVerif supports symbolic protocol analysis [4]. We relate a concrete matcher disagreement to both forms of analysis.

2. Authorization Decision Inconsistency

(1) Workflow and policy semantics.

An initiator requests contact through SAGA's Provider. The receiver's ordered policy maps identity patterns to integer budgets. Provider and receiver independently call the production matcher on the initiator. The Provider gates contact material; the receiver rechecks its policy before token issuance.

Table 1 Archived matcher observations under two rule orders.

Rule order	Expected budget	Observed budget
Specific deny, then wildcard allow	-1	10
Wildcard allow, then specific deny	-1	-1

For fixed policy P_R and valid identity I , the documented contract selects the matching rule with highest `aid_specificity()`. For its unique budget b^* , this study uses

$$\text{Allow}_{\text{spec}}(P_R, I) \iff b^* > 0.$$

Positive budgets allow and negative budgets deny. Equal-score conflicts, changing policies, and zero budgets are outside the cases.

(2) Root cause and observation.

In the matcher, `best_pattern` starts as `None` but is not updated when a matching rule changes the budget. Later rules remain compared with the initial sentinel instead of the best prior match, so a later match can replace a more specific result. Direct production-matcher calls use a specific pattern with budget -1 and specificity 70, and wildcard * with budget 10 and specificity 0.

An independent finite specification evaluator selects the specific denial for both orders. Writing

$$\text{Allow}_{\text{impl}}(P_R, I) \iff \text{match}(P_R, I) > 0,$$

the specific-deny/wildcard-allow order yields $\text{Allow}_{\text{impl}} \neq \text{Allow}_{\text{spec}}$. The reverse order returns the specified denial, establishing order dependence. Table 1 describes rule order; the D/A label in Section 4 instead denotes specification and implementation decisions. These calls establish only a local decision inconsistency, not deployment exploitation, message acceptance, or whole-system failure.

3. Formal Consequence Analysis

(1) Decision input.

Independent specification evaluation and the matcher observation supply the fixed divergence

$$(D_{\text{spec}}(P, I), D_{\text{impl}}^{\text{obs}}(P, I)) = (\text{Deny}, \text{Allow}).$$

The decision bridge reconstructs the fixed symbolic scenario

[†]TODO: affiliation and postal address.
DOI: 10.1587/trans.E0.??.

from checked evidence. ProVerif consumes fixed divergence facts; it does not execute Python or the matcher, discover the inconsistency, or prove refinement.

(2) Model assumptions.

The initiator has a valid registered identity and cannot alter the receiver’s fixed policy. The Provider, CA, effective cryptographic checks, and honest private keys are trusted. A private symbolic channel abstracts authenticated and confidential transport; symbolic state represents handoff, token issuance, and application-message acceptance. The initiator follows the prescribed symbolic peer process; this reachability result is not an arbitrary-adversary reachability claim.

(3) Queries and results.

The run reports `ChatAccept` reachable under the fixed divergence. Separately, ProVerif reports the following non-injective correspondence as true:

$$\text{ChatAccept}(R, I, t, m) \implies \text{TokenIssue}(R, I, t)$$

It requires prior issuance for the same participants and token, not intended policy permission. Issuance appears in the acceptance witness; no standalone `TokenIssue` reachability query exists.

The formal execution and consumer validation are separate experiments: the former reaches symbolic message acceptance, while the latter stops at token issuance and storage.

4. Consumer-Side Propagation Validation

(1) Purpose.

We test whether production consumers propagate matcher decisions to token issuance/storage, independently of Section 3.

(2) Environment.

The harness executes production matcher calls in Provider/access and the receiver handler, with OTK release/consumption, token generation, and `active_tokens` storage. Fixtures replace registration, certificate services, and TLS transport. Execution stops only at `receive_conversation`, after token sending and storage.

(3) Experiments.

Provider and receiver use identical policies. With specific pattern s and wildcard w , the tested policies are

$$\begin{array}{l|l} \text{A/A} & [(s, 10), (w, 10)] \\ \text{D/D} & [(w, 10), (s, -1)] \\ \text{D/A} & [(s, -1), (w, 10)]. \end{array}$$

Labels are specification/observed implementation decisions; each case ran once with fresh state and one OTK.

A/A is the positive control. D/D is Provider-blocking: HTTP 403 leaves state unchanged and never reaches the receiver, so it does not test receiver rejection. D/A tests

Table 2 Consumer propagation results; message acceptance was not executed.

Spec./impl.	Provider HTTP	OTK released	Token sent/stored
A/A	200	Yes	Yes
D/D	403	No	Not reached
D/A	200	Yes	Yes

inconsistency propagation.

(4) Result and evidence.

In D/A, the specification selects -1 . The Provider matcher returns 10; HTTP 200 releases its OTK and initializes counter 9. The receiver processes the request, returns 10, consumes the OTK, and generates, sends, and stores one token in `active_tokens`.

The empirical endpoint is token issuance/storage, not message acceptance: no application message is sent. No `receive_conversation` logic runs. Neither `local_agent.run`, LLM calls, nor tool execution occurs.

5. Scope and Conclusion

Evidence covers one recorded SAGA case and tested policies, not arbitrary policies. The symbolic abstraction establishes no Python–ProVerif refinement; controlled execution stops before message acceptance and establishes no deployed exploit. We report matcher order dependence, conditional symbolic `ChatAccept` with prior `TokenIssue`, and controlled D/A propagation to token issuance/storage.

References

- [1] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, “SAGA: A security architecture for governing AI agentic systems.” arXiv preprint arXiv:2504.21034v2, 2025. Version 2, revised 29 August 2025.
- [2] M.Y. Becker, C. Fournet, and A.D. Gordon, “Design and semantics of a decentralized authorization language,” 20th IEEE Computer Security Foundations Symposium (CSF), pp.3–15, IEEE, 2007.
- [3] K. Fisler, S. Krishnamurthi, L.A. Meyerovich, and M.C. Tschantz, “Verification and change-impact analysis of access-control policies,” 27th International Conference on Software Engineering (ICSE), 2005.
- [4] B. Blanchet, “Modeling and verifying security protocols with the applied pi calculus and ProVerif,” Foundations and Trends in Privacy and Security, vol.1, no.1–2, pp.1–135, 2016.